Real Estate Price Intelligence & Prediction System

Anand S

Tools Used: Excel, Python, Power BI, ML (CatBoost, XGBoost, DBSCAN)

Data Source: Web Scraped (MagicBricks)

Problem Statement

Build a system that helps understand price drivers and accurately predict property prices because:

- Real estate pricing is **highly inconsistent and opaque**
- Buyers struggle to Identify fair pricing
- Buyers struggle to compare price across locations
- Sellers struggle with Competitive pricing
- Platforms lack **data-driven pricing intelligence**

Objectives

Develop a data-driven system to analyse key real estate price drivers, identify market trends, and accurately predict property prices for informed decision-making.

- Analyse **key factors influencing property prices**
- Identify **high-value and low-value locations**
- Build **predictive models for price estimation**
- Enable **interactive exploration via dashboard**
- Support **data-driven decision making**

Video presentation

link

https://docs.google.com/videos/d/1c_TvxldXTilQiVd7uUhY6DZ8-LMRxTgKaUovD4ZdbuM/edit?usp=drive_link

Data Collection (Web Scraping)

Source: MagicBricks

Method: Web scraping using Python.

Used Selenium to automate browsing and load dynamically rendered content via infinite scroll. Once the page was fully loaded, BeautifulSoup was used to parse the static HTML and extract relevant property details.

Data collected:

- City, title, price, sqft price,
- property name, summary,
- posted by, description

Challenges:

- Missing values
- Inconsistent formats
- Outliers
- Extraction of valuable features like, location, BHK info, furnishing, amenities etc. from collected data

Data Preparation & Feature Engineering

Data Cleaning & Preparation

- Standardized categorical values (lowercase, trimmed inconsistencies)
- Cleaned numerical fields (bathroom, balcony) by removing non-numeric characters
- Handled missing values using:
    - Median (numerical features)
    - Mode (categorical features)
    - Logical defaults (e.g., parking = 0, ready_to_move = 0)
- Handled missing values of some categories into '**insignificant/other/missing**' wherever appropriate
- Removed irrelevant and redundant columns
- Eliminated incorrect geolocation entries

Feature Engineering

Extracted and engineered 24 features from the primary 7 columns collected from MagicBricks.

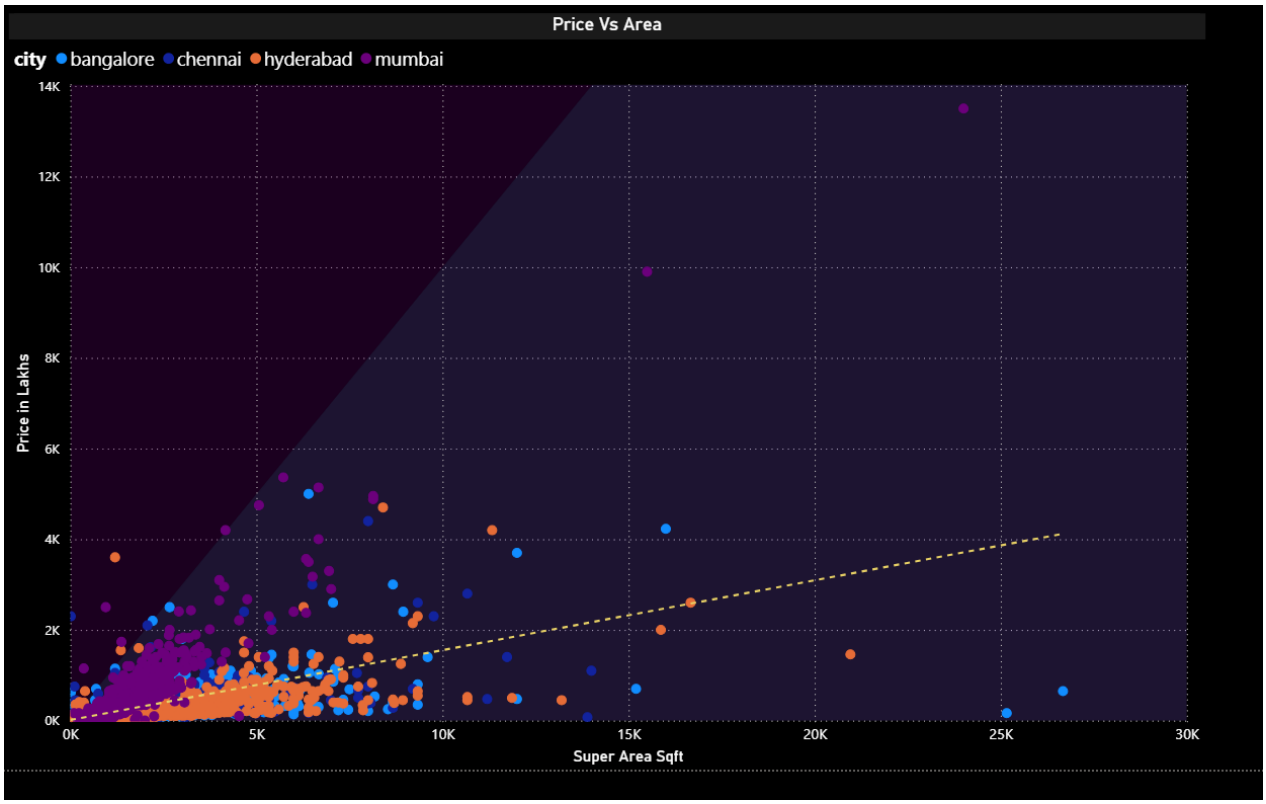
- Extracted structured features (e.g., *facing, overlooking*) from unstructured text descriptions
- Parsed complex *floor* data into **floor number** and **total floors**
- Created **floor ratio** and derived **floor category (low/mid/high)**
- Engineered parking features:
    - Total parking, Covered and open parking.
    - Parking availability (binary)
- Generated **area per BHK** for better space utilization analysis
- Applied **log transformation on price** to handle skewness
- Created **price segmentation**:
    - Pan-India (Budget → Luxury)
    - City-level segmentation
- Generated **binary indicators** (e.g., has_overlooking)

Geospatial Enrichment

- Retrieved latitude & longitude using geocoding
- Implemented caching to avoid repeated API calls
- Imputed missing coordinates using **city-level average**

EDA and key insights

1. Area vs Price



While property price increases with area, significant variability across similar sizes indicates that location and market factors play a more dominant role than size alone.

Majority of data points are concentrated in:

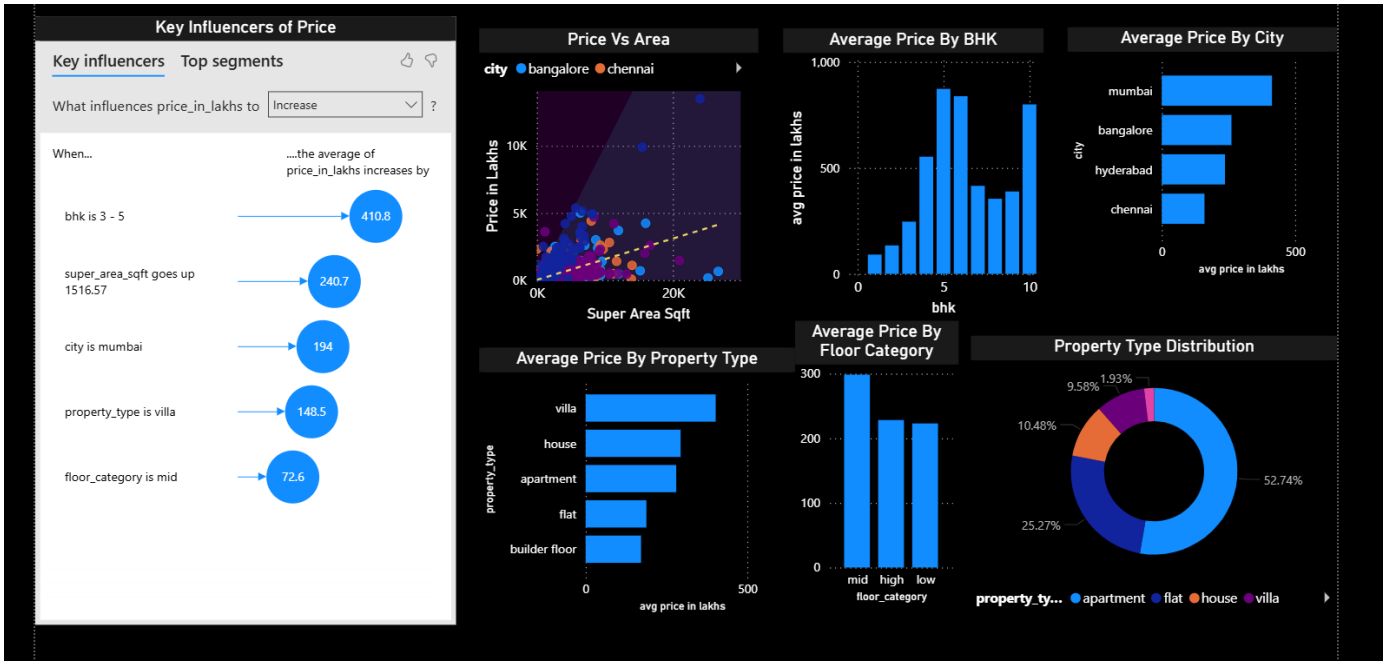
- **< 6,000 sqft**
- **< 2,000 lakhs**

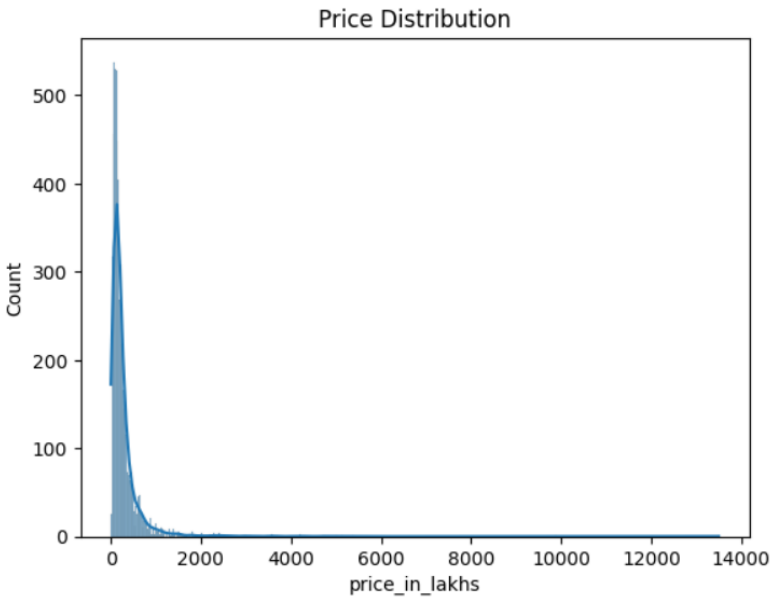
Indicates the dataset is dominated by **mid-sized, mid-priced properties**

Some cities (likely Mumbai) show **higher price concentration even at lower areas**

Suggests **location premium outweighs size in certain cities**

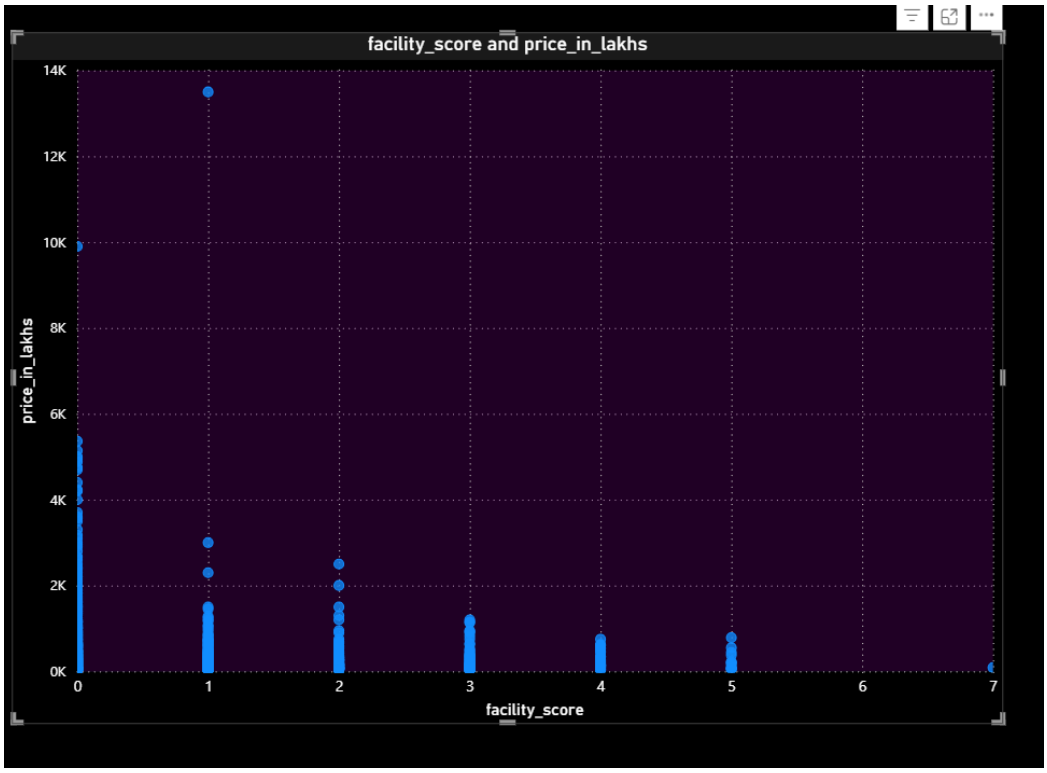
2. Price distribution and analysis





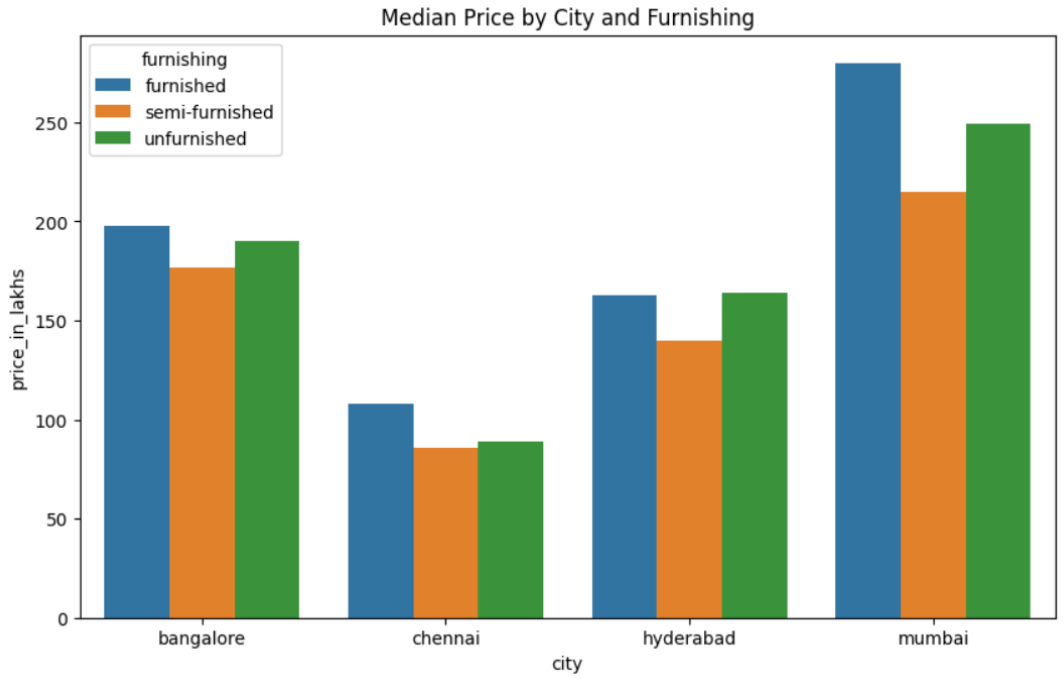
- **Histogram of price: Right skewed, extreme outliers.**
- **City: Mumbai- high prices, Chennai – lower prices**
- **Property Type: villa and houses – high, builder floor and flat – low**
- **Middle floors – higher prices**

3. Facility Score vs Price



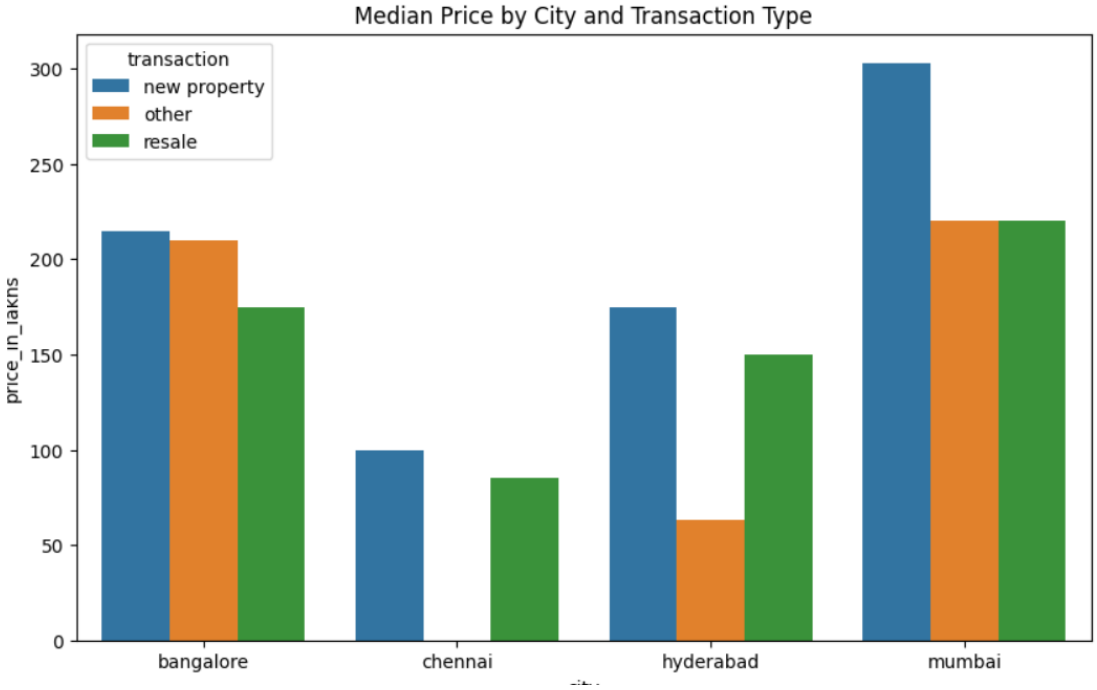
- Scatterplot between facility score and price
- **Insight:** The facility score feature did not provide sufficient data for a meaningful analysis. Although the scatter plot suggests a slight inverse relationship—where prices appear to decrease as facility scores increase—this pattern is likely unreliable and may be driven by limited or incomplete information on nearby facilities

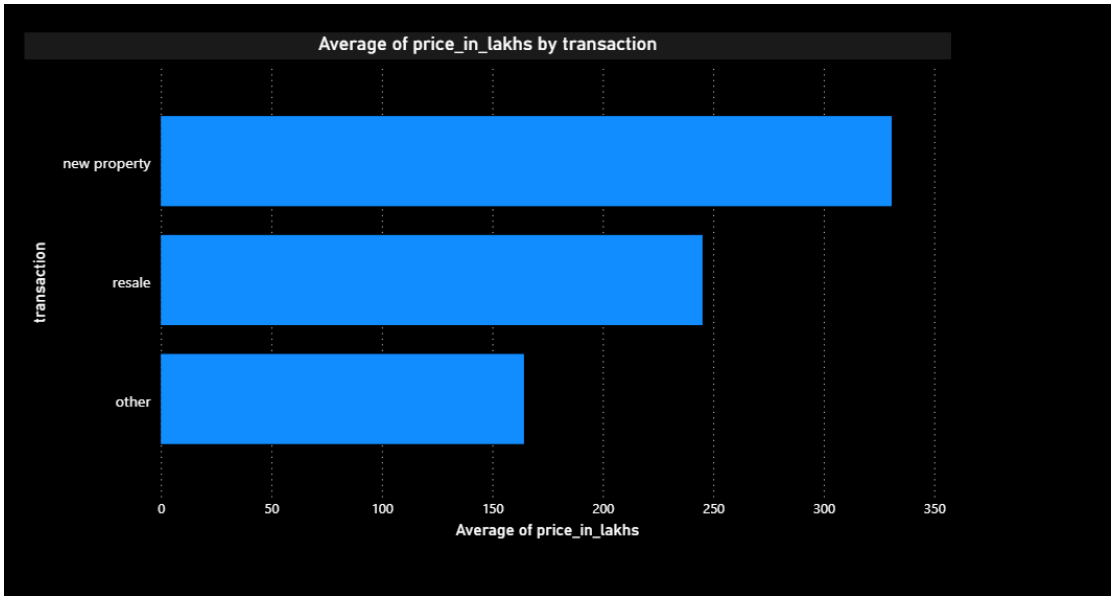
4. Furnishing Impact



- Compared median prices across furnishing types
- **Insight:**
  - Furnished properties consistently priced higher
  - Strong effect across all cities (especially Mumbai)

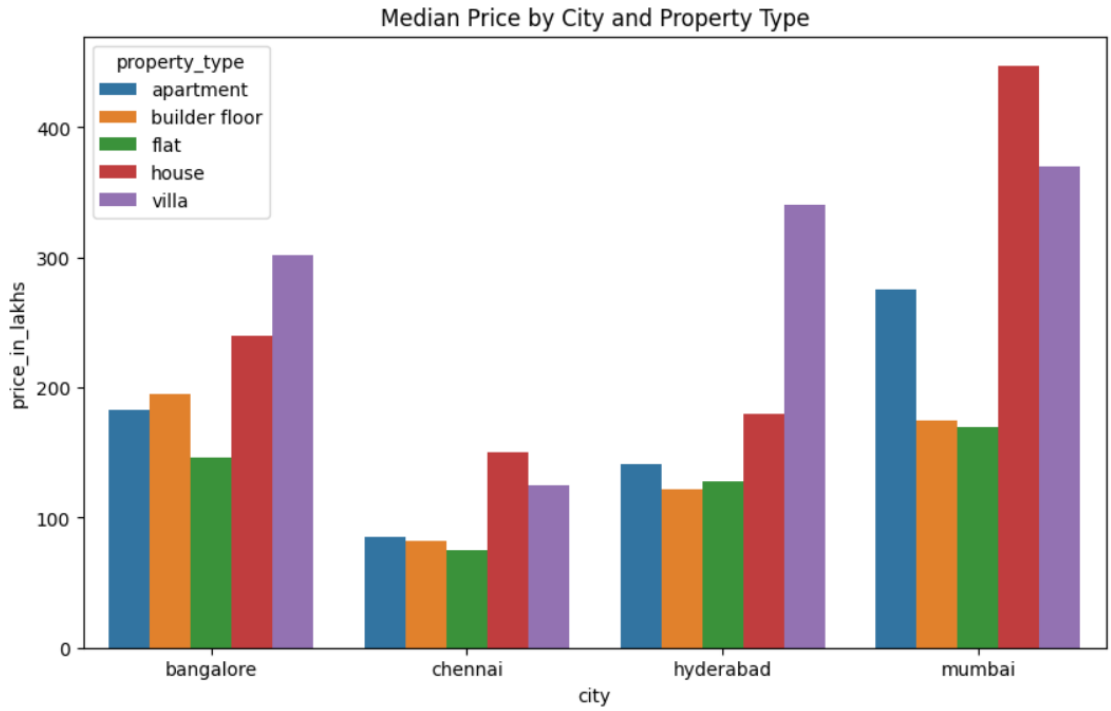
5. Transaction Type (New vs Resale)





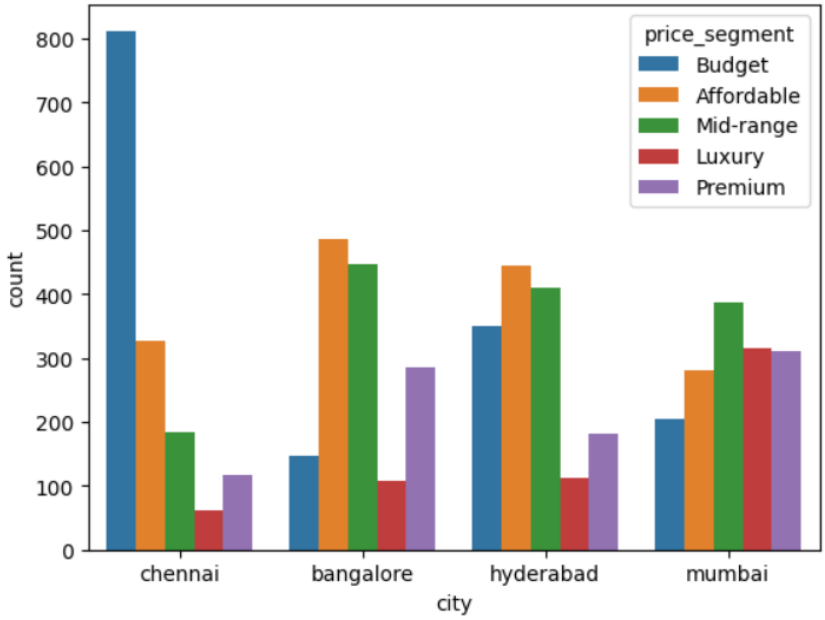
- Average Price by transaction type
- Compared median price by city and transaction type
- **Insight:**
    - New properties priced higher than resale
    - Premium gap especially large in Mumbai

6. Property Type Analysis



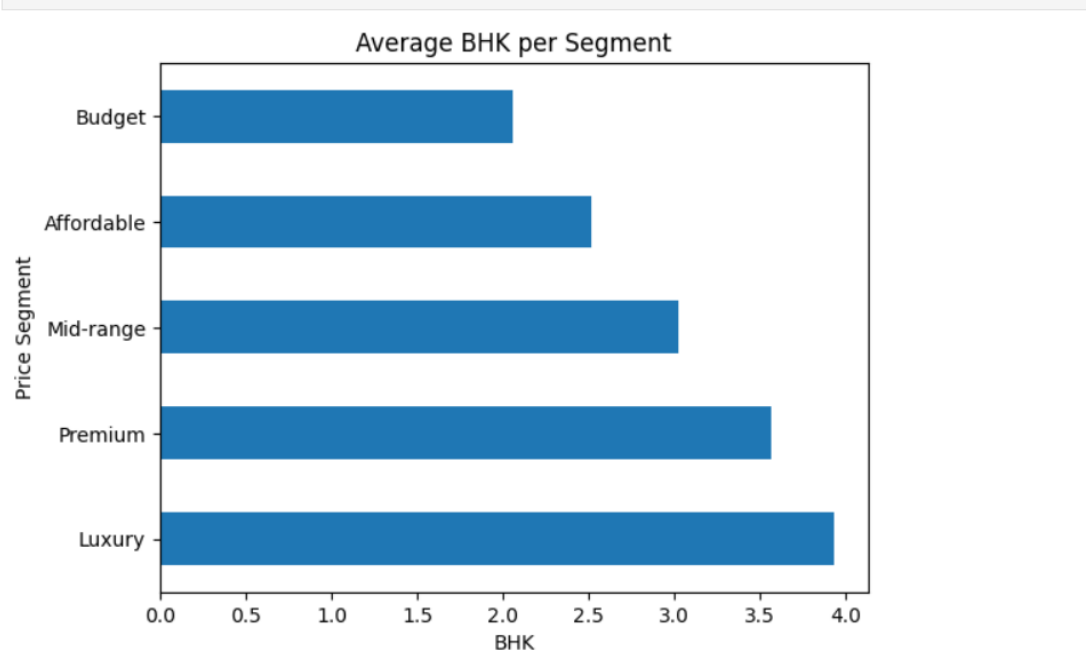
- Compared price across property types
- **Insight:**
    - Villas & independent houses are most expensive across cities

7. City-wise Price Segmentation

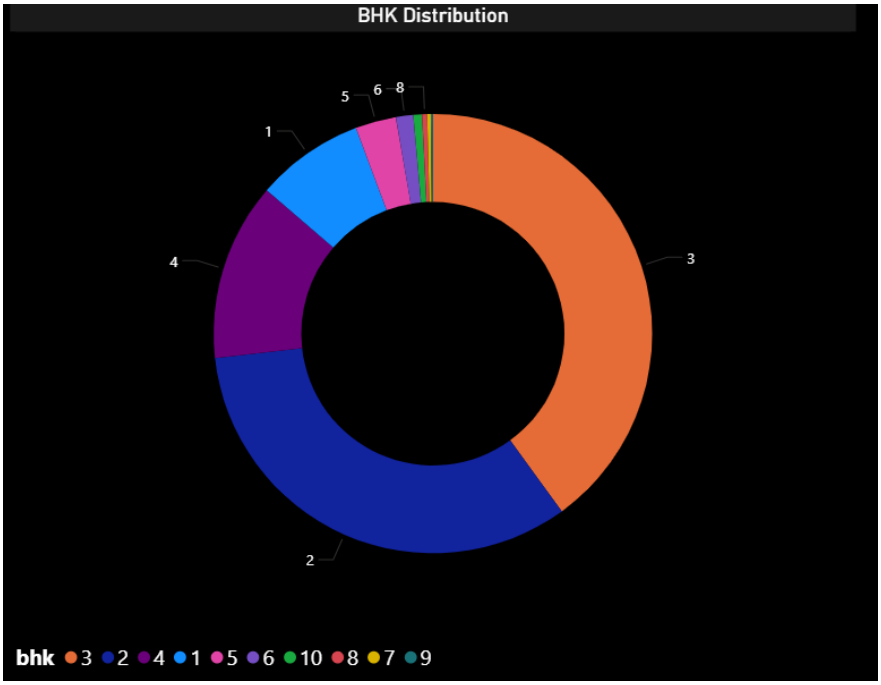


- Countplot of price segments across cities
- **Insight:**
    - Chennai has higher share of budget properties
    - Other cities skew towards mid/premium segments

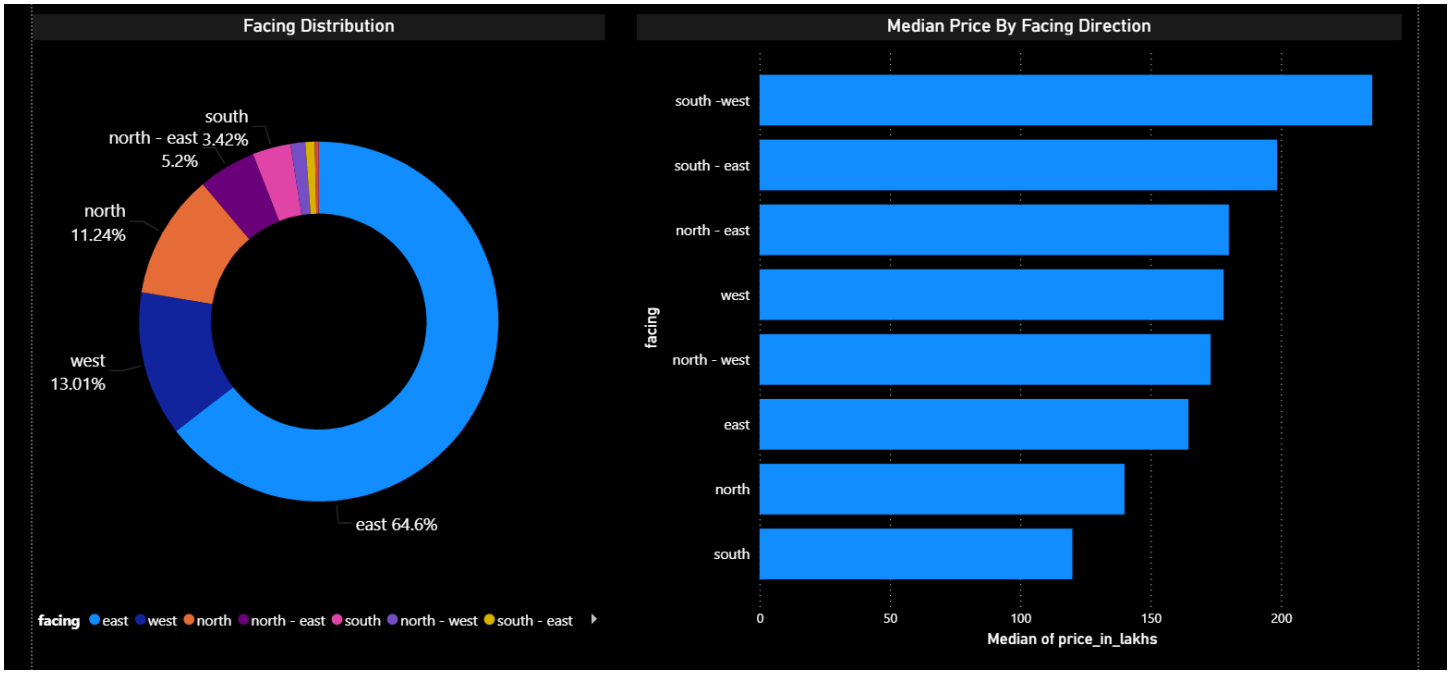
8. BHK vs Price Segment



- Average BHK across price segments
- **Insight:**
  - Higher price segments → larger BHK
  - Confirms size as key price driver
  - The BHK distribution indicates that most of properties fall within 1-4 BHK range.



9. Facing vs Price



- East-facing properties dominate the dataset, accounting for **~65% of listings**
- West and North-facing properties form the next largest segments, but are significantly lower (~13% and ~11%)
- Other directions (South, North-East, etc.) have **minimal representation**, indicating limited supply
- **South-West facing properties command the highest median prices**, indicating a premium preference
- **South-East and North-East** also show relatively high median prices
- Despite being the most common, **East-facing properties are mid-range in pricing**
- **South-facing properties have the lowest median prices**, indicating weaker demand or preference

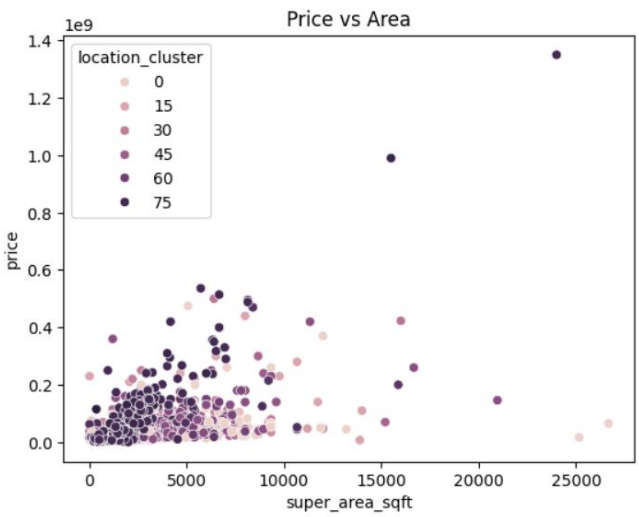
Model Building

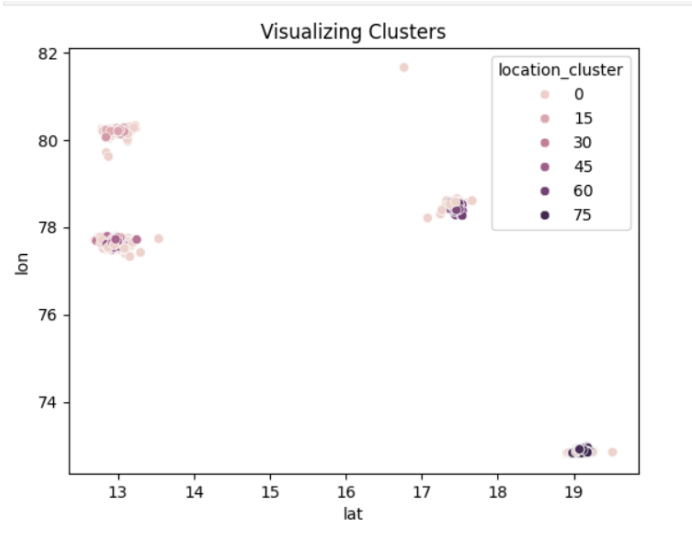
Location Clustering using DBSCAN

To capture location-based patterns, a **DBSCAN clustering model** was applied on property coordinates (latitude and longitude). The coordinates were converted to radians and the **Haversine distance metric** was used to accurately measure geographic distances.

An epsilon distance of approximately **1.5 km** and a minimum sample size of **10** were used to identify dense clusters of properties. The model grouped nearby properties into distinct **location clusters**, while isolating sparse or unique locations as noise.

The resulting location_cluster feature was then integrated into the dataset and used as an additional input for prediction models, enabling them to better capture **micro-location effects** on property pricing.





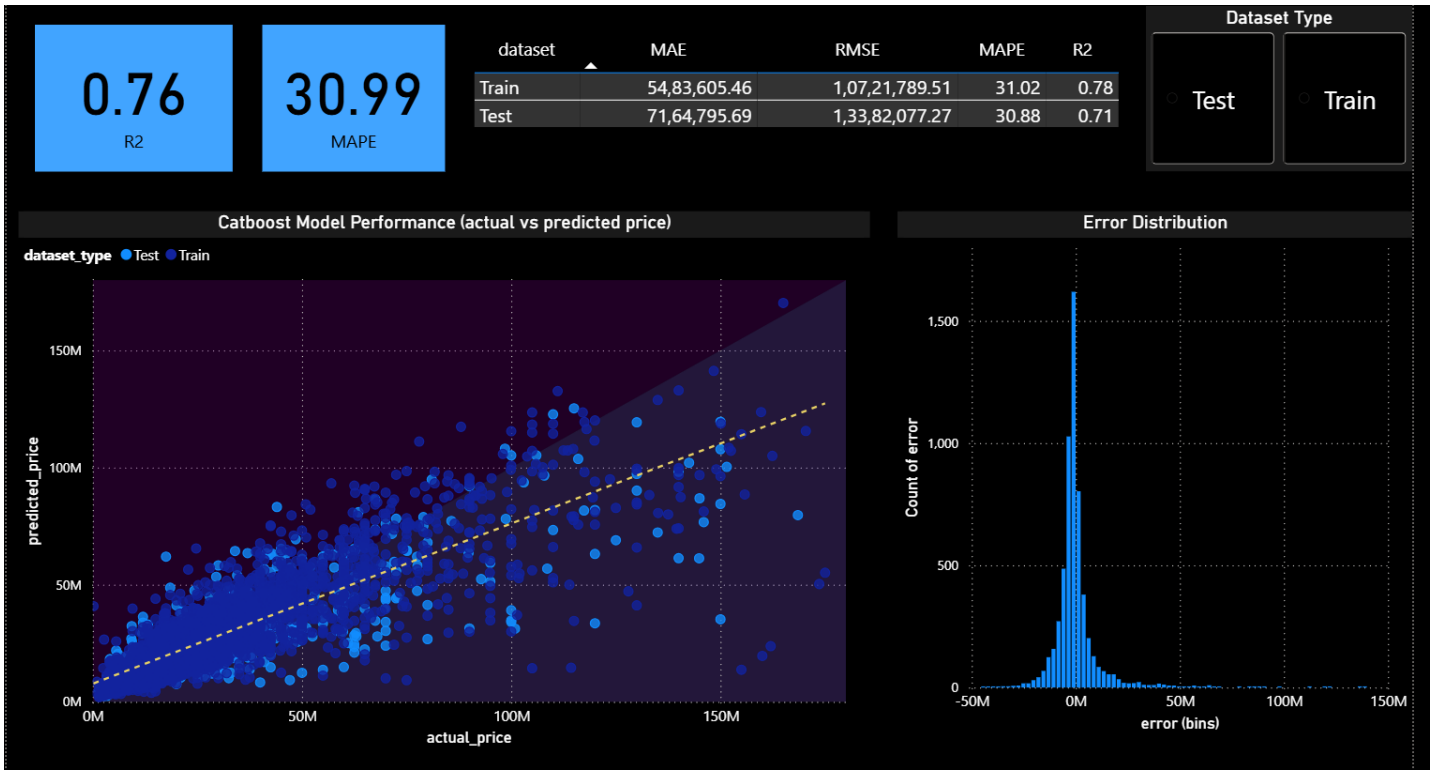
CatBoost Model Development & Optimization

- **Data Preparation**
  - Removed leakage features and extreme outliers (top 1%)
  - Applied **log transformation** on target (price) to handle skewness
  - Split data into **train (80%) and test (20%)**
- **Hyperparameter Tuning (Optuna – Pre-Training)**
  - Used **Optuna (Bayesian optimization)** *before final model training* to identify optimal parameters
  - Tuned: iterations, depth, learning_rate, l2_leaf_reg, random_strength etc.
  - Objective: maximize **R^2 score** on validation data
  - Best parameters from Optuna were stored and reused
  - `Best Params: {'iterations': 1375, 'depth': 7, 'learning_rate': 0.09627105358413003, 'l2_leaf_reg': 1.3342176171847173, 'bagging_temperature': 0.9762780896850152, 'random_strength': 8.061297966615442, 'border_count': 125, 'rsm': 0.897261620767895}`

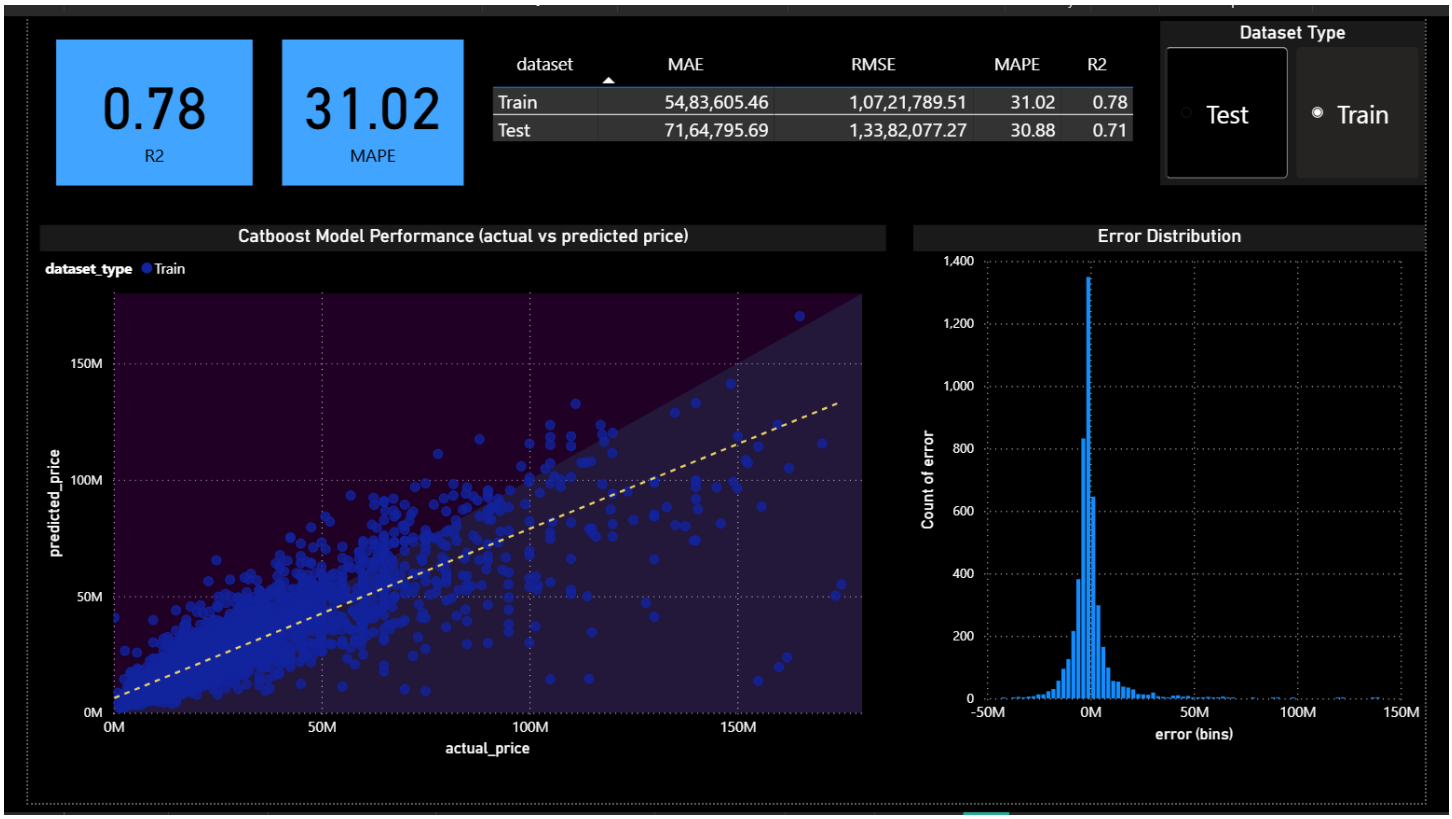
- **Model Building (CatBoost with Tuned Parameters)**
  - Initialized CatBoost using **best parameters discovered by Optuna**
  - Leveraged native handling of **categorical features**
  - Trained with **early stopping** using validation set to prevent overfitting
- **Model Evaluation**
  - Predictions inverse-transformed to original scale
  - Evaluated using:
    - **R^2** → model fit
    - **MAE / RMSE** → error magnitude
    - **MAPE** → percentage error

Model performance visualized in Power BI :

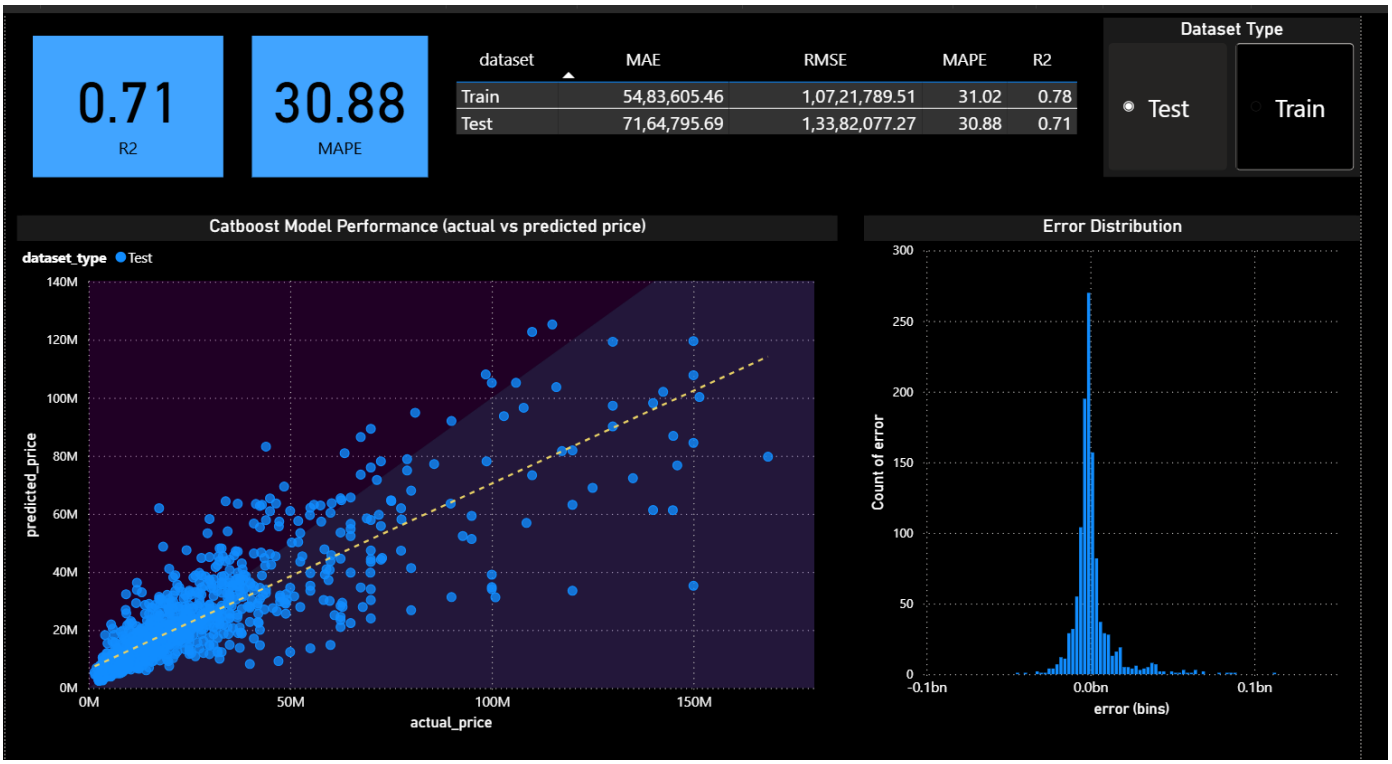
Overall Model Performance



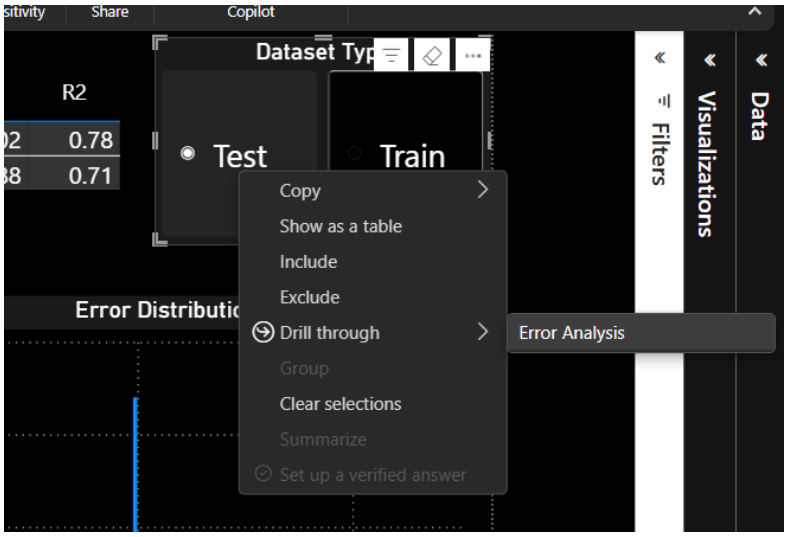
Performance during Training



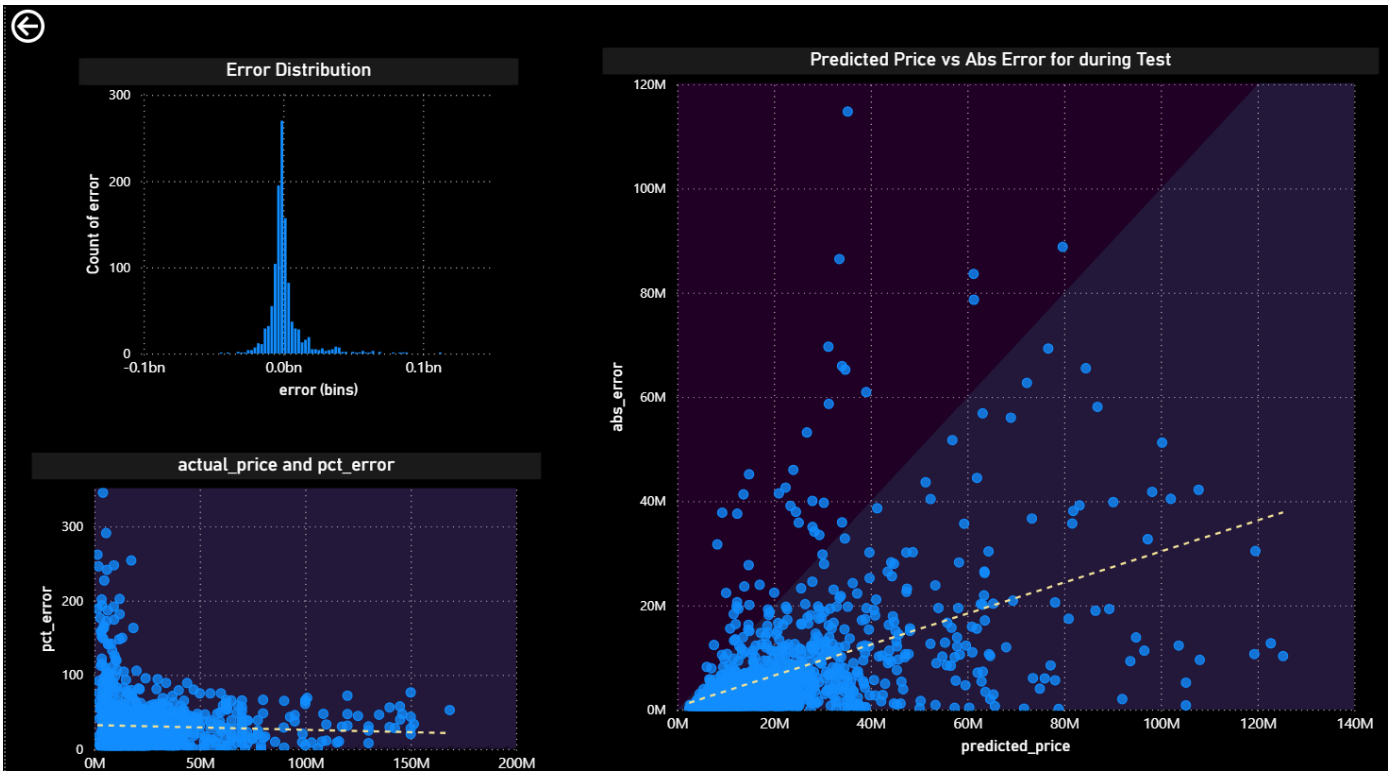
Performance during Testing



Moving to detailed error analysis for test dataset using drill through feature of Power BI



Error analysis (Test dataset)



The model performs reasonably well but is constrained primarily by **data quality, missing contextual features, and distribution imbalance**, rather than model capability alone

- Model achieved **$R^2 \approx 0.71$** and **MAPE ≈ 30.9%**, indicating moderate predictive performance
- Errors are centered but show **high variance with notable outliers**
- **Higher % errors in lower price ranges**, while **absolute errors increase for high-value properties**

Key Reasons for Error

- **Data quality issues** from web scraping (missing, inconsistent, unstructured data)
- **Limited feature depth** (missing factors like locality quality, connectivity, builder reputation)
- **Imbalanced data distribution** with fewer extreme price points
- Model struggles with **extreme values and price variability across locations**

Attempted improvement

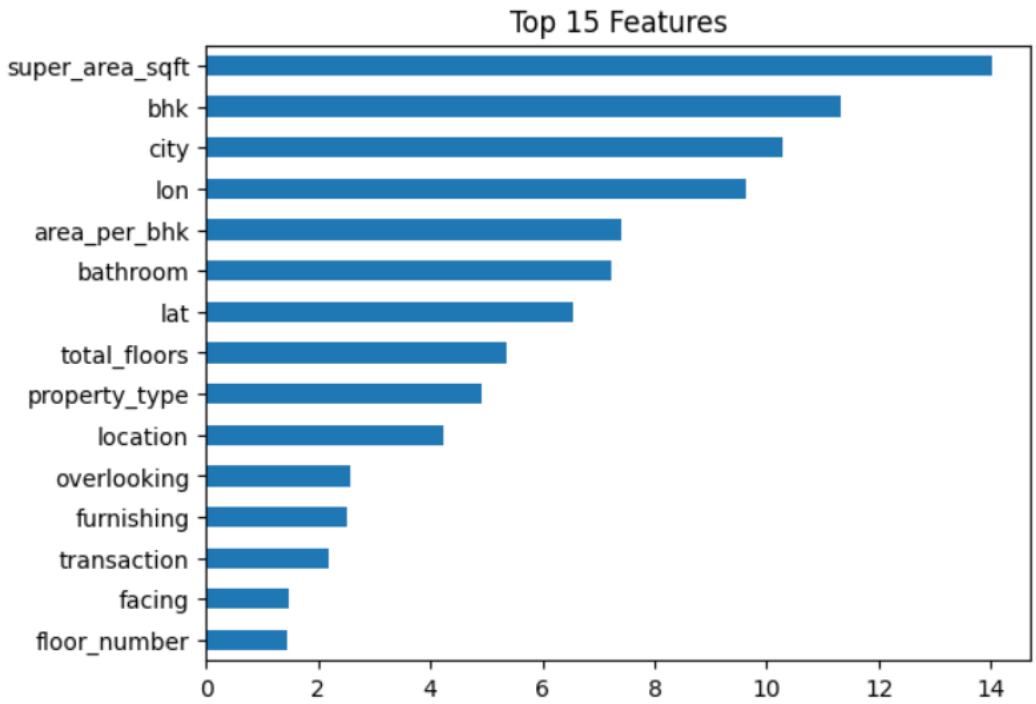
- **Used MAE (via RMSE calculation on validation set) as the objective during hyperparameter tuning to directly minimize prediction error**
- **Set Optuna study to minimize error instead of maximizing R^2**
- **Kept RMSE as both loss function and evaluation metric during model training**
- **This approach focused tuning on reducing error magnitude, while training remained consistent with RMSE optimization**
- **Result: marginal improvement in error metrics, with overall model performance largely unchanged**

```
--- Train Performance ---
R2: 0.8248410090945836
MAE: 4990696.656722231
RMSE: 9561387.801793572
MAPE(%): 22.744153117381284

--- Test Performance ---
R2: 0.6983647999962391
MAE: 7118338.014233375
RMSE: 13544112.314722814
MAPE(%): 29.075053333157193

Train R2: 0.8248410090945836
Test R2: 0.6983647999962391
```

Improvement Opportunities

- Enhance **data collection & standardization** for cleaner inputs
- Add **location intelligence & external features** (infrastructure, demand indicators)
- Apply **better outlier handling**
- **Segment-based modeling** is recommended to address varying pricing patterns across different property ranges, as a single model struggles to accurately capture the distinct dynamics of low, mid, and high-priced properties, leading to higher prediction errors.
- Explore **ensemble models and advanced hyperparameter tuning**

Top 15 Features According to CatBoost Model



XGBoost Model Development & Optimization

Built using log-transformed target, with OneHotEncoding for categorical features and a preprocessing pipeline.

Optuna tuning (30 trials) optimized key parameters (depth, learning rate, regularization), using cross-validated R^2 as the objective.

Final model shows:

```
--- Train Performance ---
R2: 0.802123489366849
MAE: 5295189.206717524
RMSE: 10162529.928268297
MAPE(%): 24.48189274656126

--- Test Performance ---
R2: 0.7016139766335794
MAE: 7235526.806779661
RMSE: 13470967.061826972
MAPE(%): 30.216982913052448

Train R2: 0.802123489366849
Test R2: 0.7016139766335794
```

Train R^2: ~0.80, Test R^2: ~0.70

Moderate gap → some overfitting, but not severe

MAPE ~30% (test) indicates consistent relative error across predictions

Key Inferences

Model generalizes reasonably well, with stable test performance

Slight overfitting suggests model captures training patterns better than unseen data

Error metrics (MAE, RMSE, MAPE) are similar to CatBoost, no major improvement

Comparison & Conclusion

XGBoost and CatBoost perform very similarly on test data

CatBoost shows:

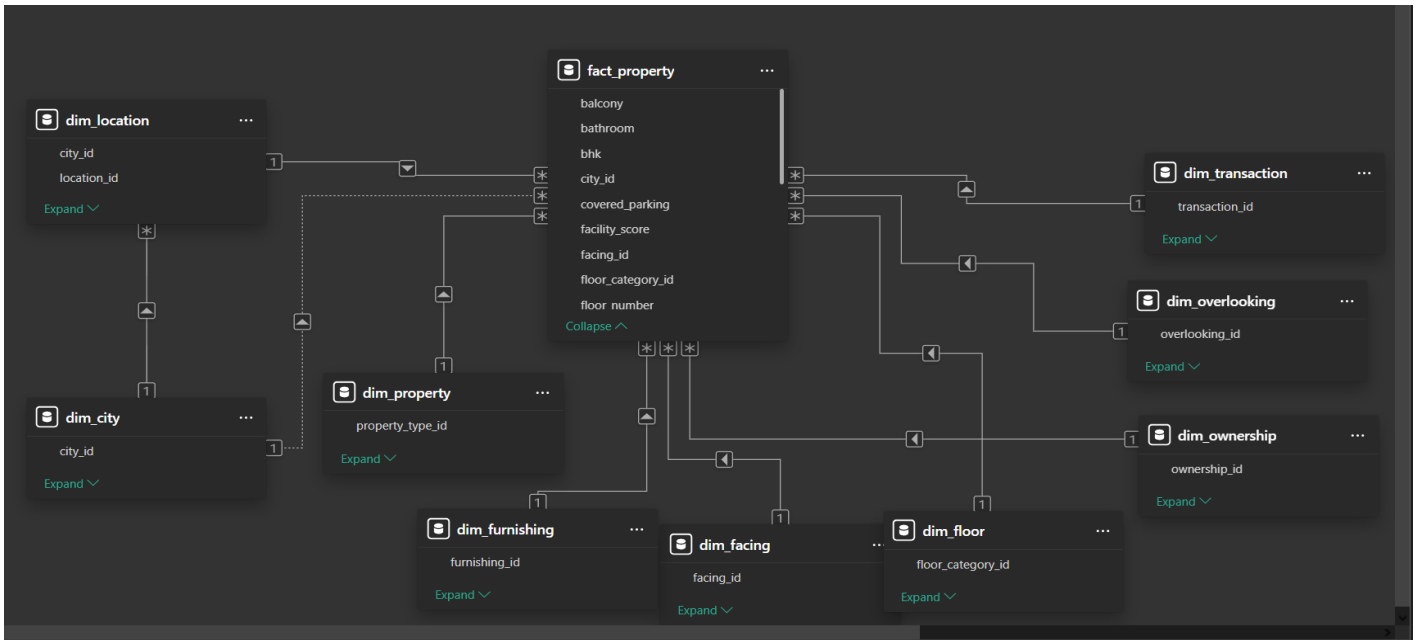
Slightly lower error (MAPE, MAE)

Better handling of categorical features (no encoding needed)

Conclusion:
CatBoost is marginally better due to slightly lower error and simpler pipeline, while both models show comparable generalization performance.

Power BI Dashboard

Modelling

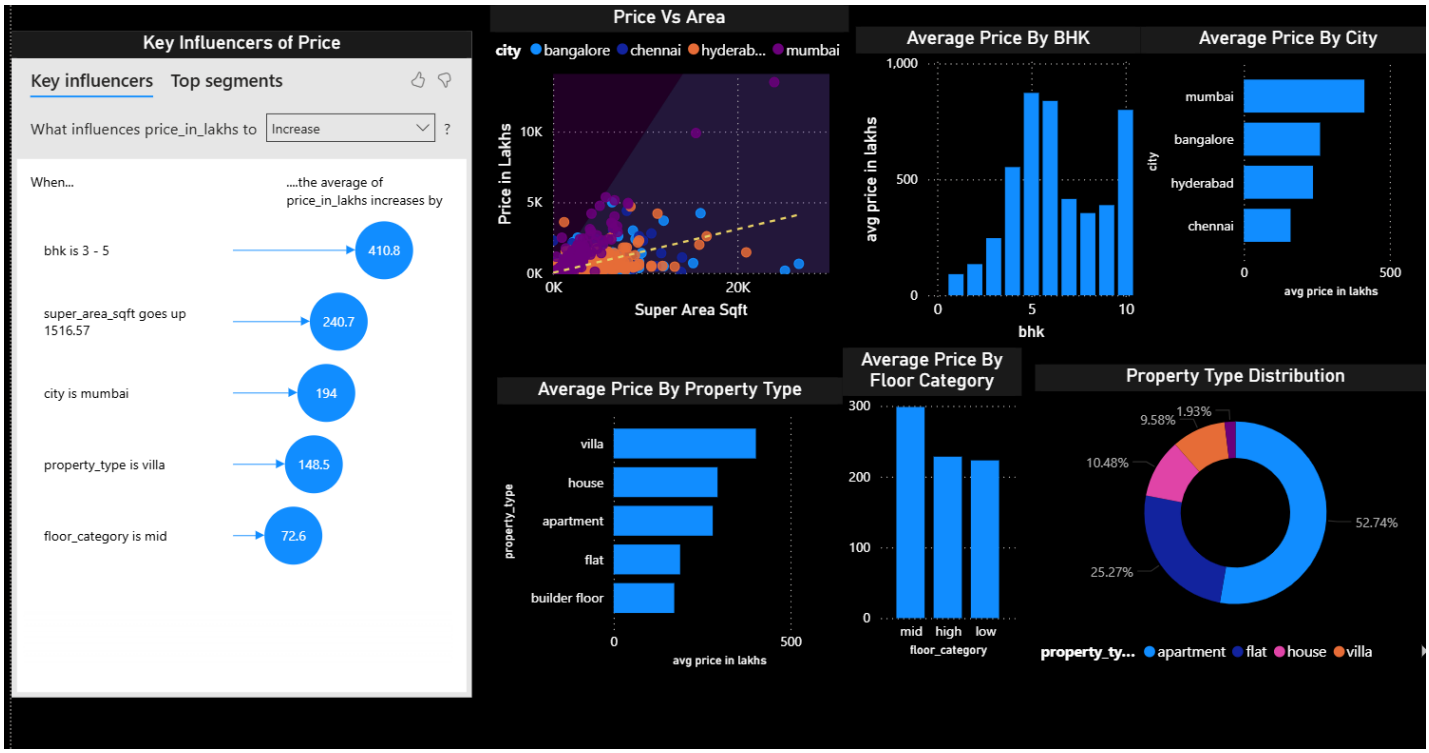


Fact table: Fact property

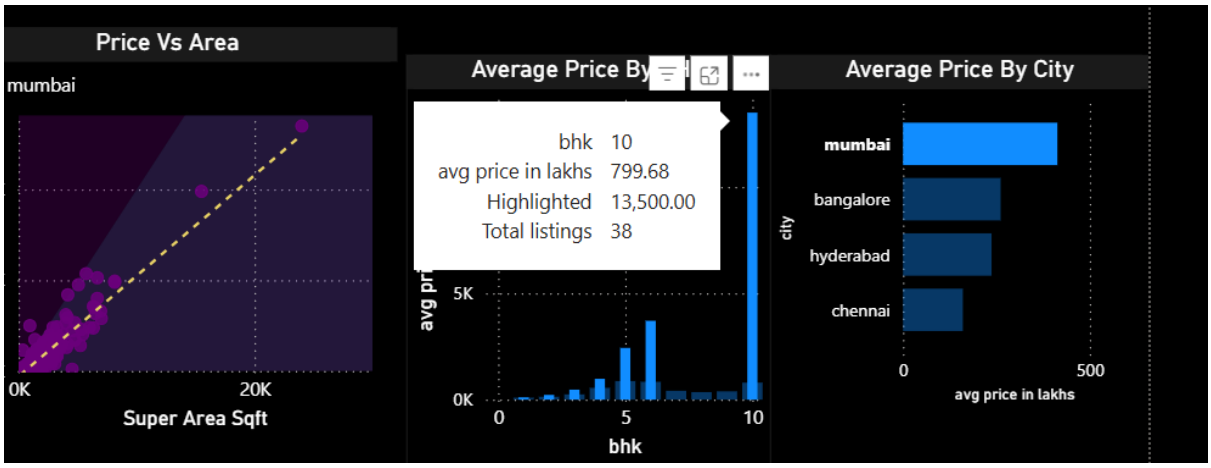
Dimension Tables:

- Dim_location
- Dim_city
- Dim_property
- Dim_furnishing
- Dim_facing
- Dim_floor
- Dim_transaction
- Dim_overlooking
- Dim_ownership

Key Influencers of Price

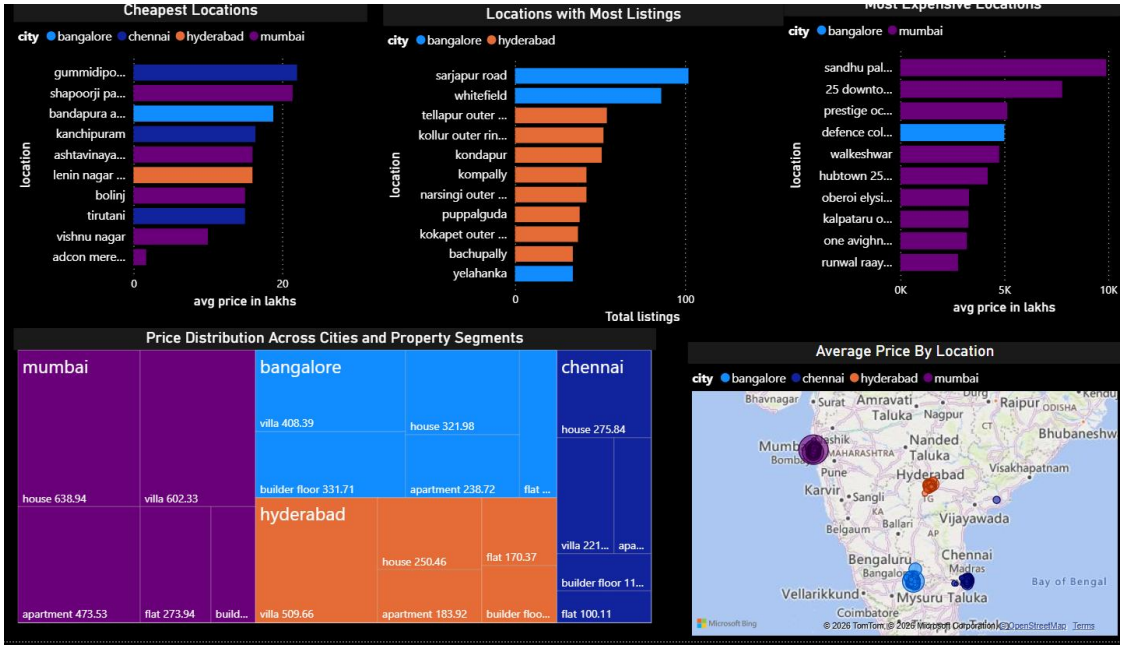


Pricing is primarily driven by a combination of **size, configuration (BHK), location, and property type**, with **clear alignment between influencer outputs and visual EDA trends**, validating the insights.



- In Mumbai, price exhibits a strong linear relationship with area, indicating that **area is a key driver of property prices**.

Location Based Insights



- **Map visualization shows clear city-wise clustering**, with Mumbai dominating high-value clusters, Bangalore in the mid-range, and Hyderabad/Chennai forming lower-priced clusters.
- **Treemap analysis highlights price distribution across cities and property types**, showing that **villas and houses drive premium pricing**, especially in Mumbai.
- **Most expensive locations are concentrated in Mumbai**, reinforcing its position as the highest-priced market.
- **Cheapest locations are spread across cities**, indicating availability of affordable options in multiple regions rather than a single city.
- **High listing activity is localized**, with certain micro-markets contributing disproportionately to total supply, signaling demand hotspots.

Location Deep Dive

Page accessed through selecting a location and drilling through

d

Location Deep Dive page:

The page provides a **granular view of a selected location**, combining key metrics, listing-level details, and spatial insights.

A **multi-row card** highlights important KPIs such as **city, total listings, average price, and luxury ranking (overall and within city)**, enabling quick benchmarking.

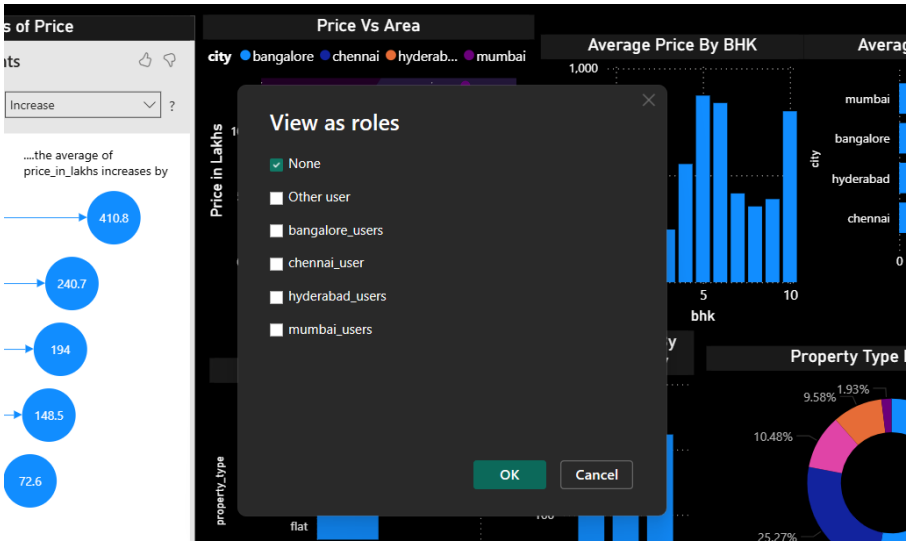
A **price range slicer** allows dynamic filtering, helping analyze how listings and pricing change across budget segments.

A **detailed listings table** presents property-level information, with **data bars on price and area** to visually compare listings and identify high-value properties.

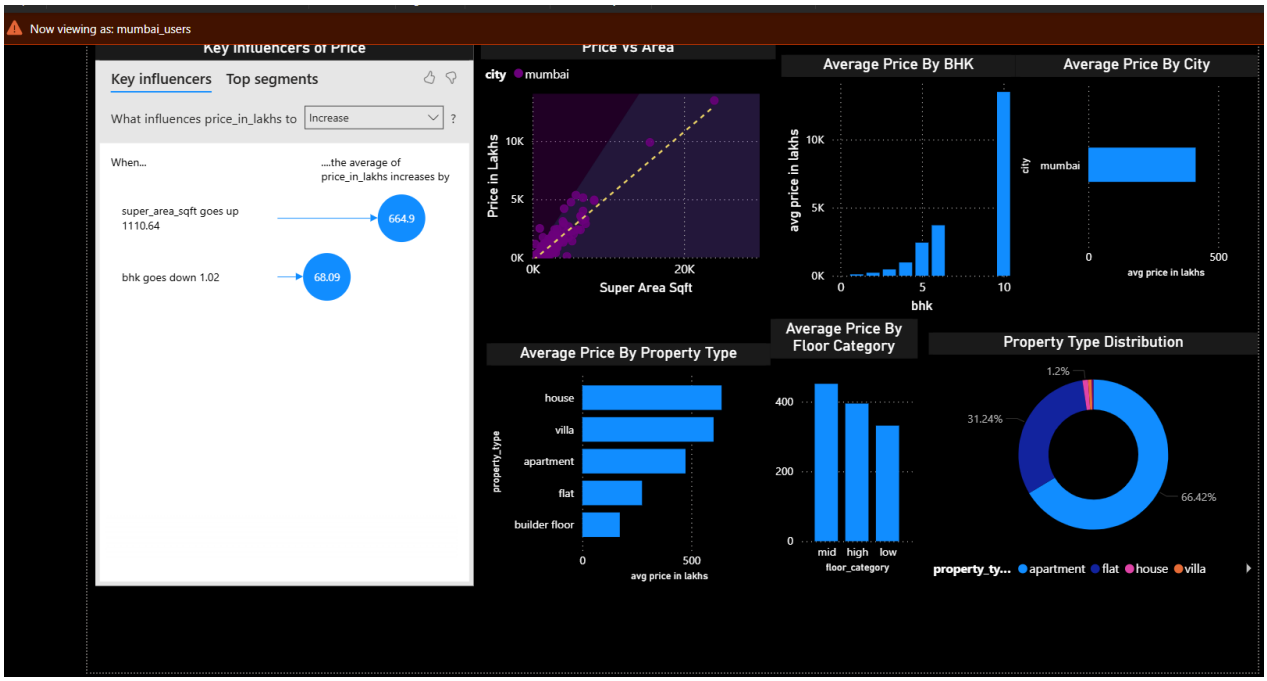
The **map visualization pinpoints the exact location**, with **bubble size representing price** and **city-based legend**, providing geographic context and price concentration.

Price Drivers by Property Features: Bar charts showing average price variations by BHK, furnishing status, and floor category, providing insights into how property characteristics impact pricing for this location

Row Level Security

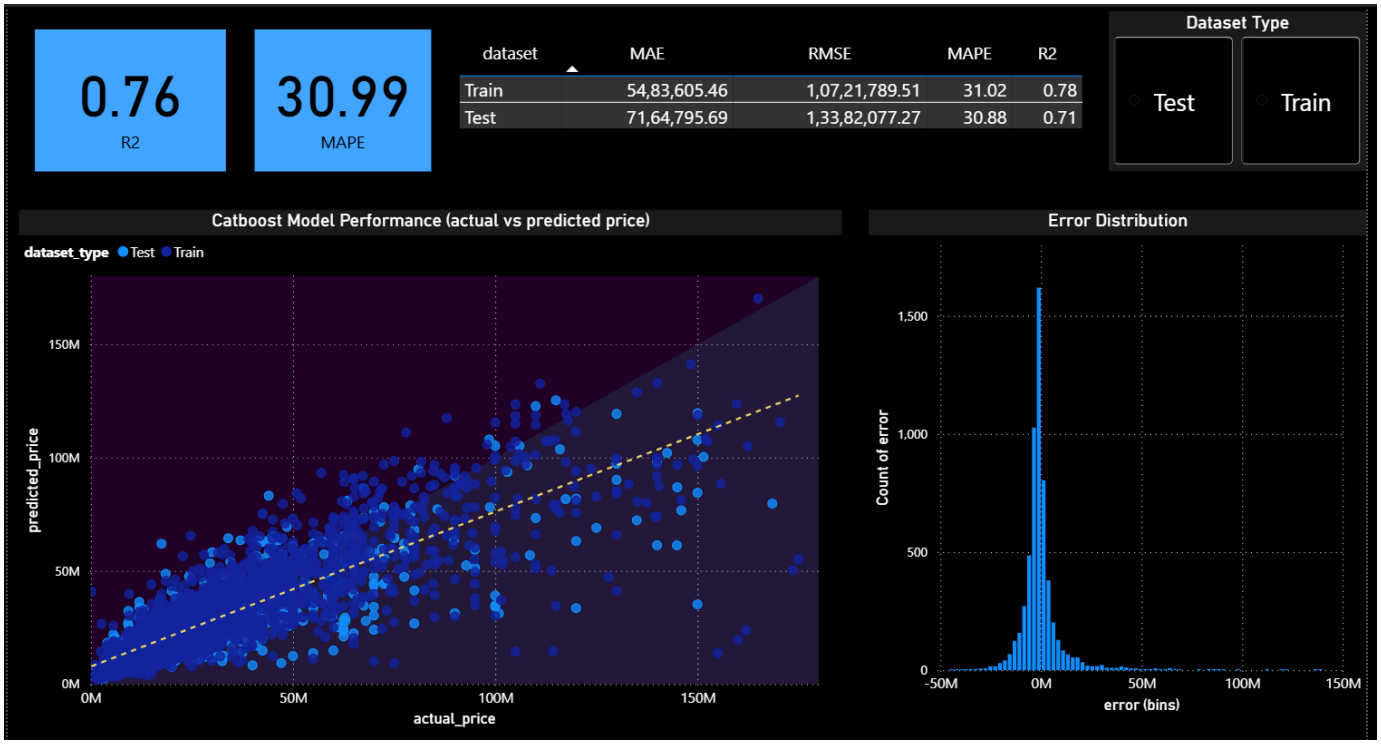


Row-Level Security (RLS) was implemented in Power BI Desktop by creating **city-based roles** that filter data at the row level. Roles were defined for **Bangalore, Chennai, Mumbai, and Hyderabad users**, each restricted to viewing data for their respective city. These roles can be assigned to users in Power BI Service based on their location, ensuring controlled and relevant access to data.



View of Mumbai users

Model Performance



The model performance page provides a **consolidated view of evaluation metrics**, with KPI cards highlighting **R^2 and MAPE** for quick assessment.

The model achieves **R^2 of 0.78 (train) and 0.71 (test)**, indicating **good explanatory power with limited overfitting**.

MAPE is ~32% (train) and ~31% (test), suggesting a **moderate level of prediction error**, consistent across datasets.

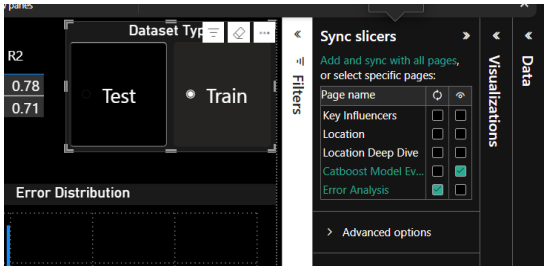
A **detailed metrics table** presents multiple evaluation measures for both train and test sets, enabling deeper comparison.

An **error distribution bar chart** visualizes how prediction errors are spread across ranges, helping identify concentration of inaccuracies.

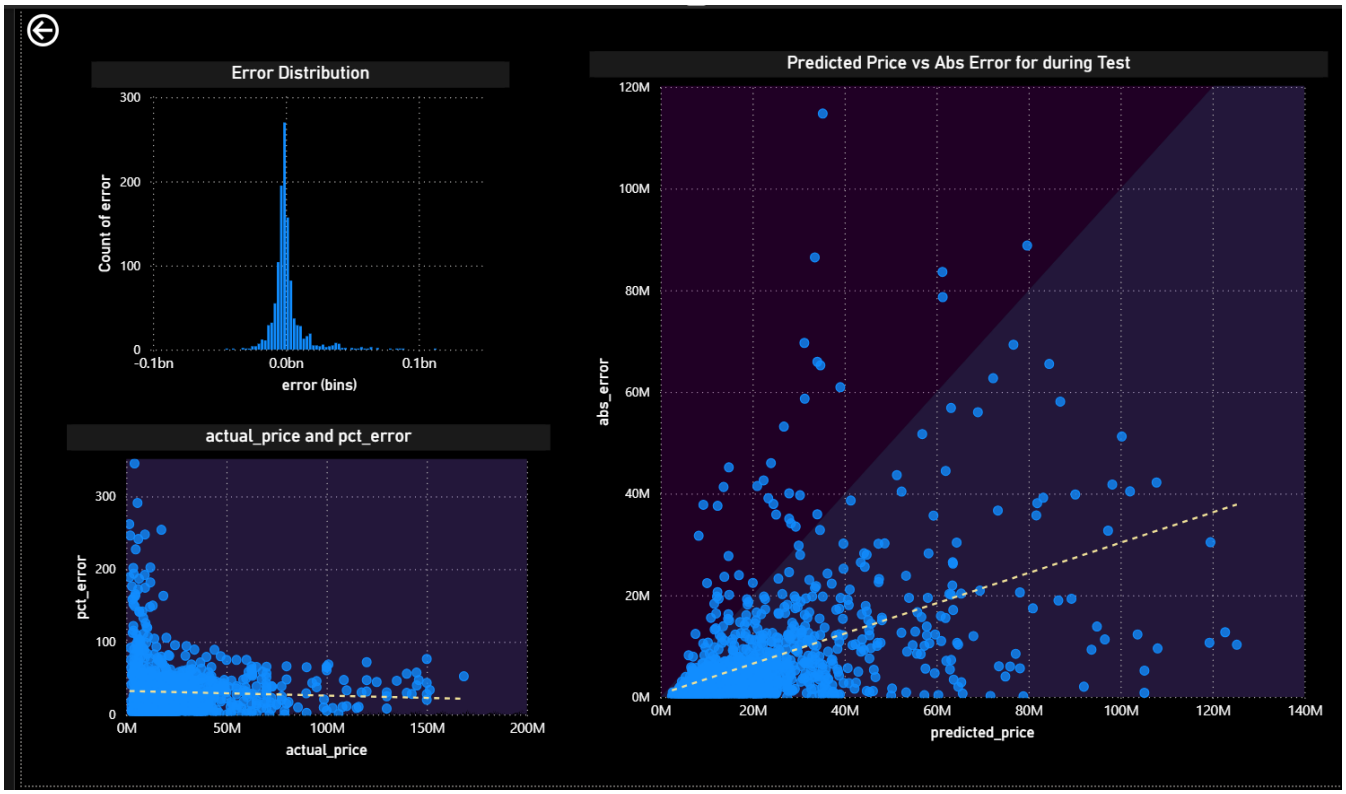
A **slicer (synced across pages)** allows interactive filtering of error ranges, which also drives a dedicated **error analysis page**.

Drill-through functionality enables users to move from summary metrics to granular error-level insights for deeper investigation.

Sync slicer



Error Analysis



Error Distribution (Bar Chart):
Shows how prediction errors are spread across predefined ranges, helping identify where errors are most concentrated and whether extreme errors (outliers) are significant.

Scatter Plot – Predicted Price vs Absolute Error:
Used to examine how error magnitude changes with predicted values, highlighting whether higher-priced predictions lead to larger absolute errors.

Scatter Plot – Actual Price vs Percentage Error:
Helps analyze relative error behavior, revealing that lower-priced properties tend to have higher percentage errors, while higher-priced properties show comparatively stable % error.

Final Recommendations

Based on the analysis:

- Segment-based modeling could improve prediction accuracy
- More consistent and structured data collection would reduce noise
- Additional features like amenities and neighborhood quality could enhance performance
- The dashboard can be used to identify investment opportunities and pricing inefficiencies

Video Presentation

https://docs.google.com/videos/d/1c_TvxldXTilQiVd7uUhY6DZ8-LMRxTgKaUovD4ZdbuM/edit?usp=drive_link

link